

JS Variables

1. Javascript Variable:

- a. A variable is a container that stores data values.
- b. They are containers for storing data.
- c. Variable is nothing but the container, where we can store the value or data.

2. Identifier vs Variable:

- a. **Identifier** → A name given to variable, function, or other items.

For Example → let age; // 'age' is an identifier

- b. **Variable** → A variable is the storage location which is identified using an identifier.

For Example → let age = 25; // 'age' is a variable with the value 25.

3. Declaring Variables in JavaScript:

- a. To use a variable in JavaScript, you must first declare it.
- b. This means creating a variable using specific keywords: **var**, **let**, or **const**.
- c. Below is a detailed explanation of each type of declaration.
- d. There are 3 declaration types in JavaScript:
 - i. var
 - ii. let
 - iii. const

1. **var** ⇒

- a. The var variable is the oldest way to declare variables in JavaScript.
- b. Variables declared with var have function scope or global scope, depending on where they are declared.
- c. Variables declared with var are hoisted.
- d. **Disadvantage of var variable** → **Variable Pollution**
- e. For Example →

```
1  //! 1. Declaration ⇒ var: You can declare a variable using var
2  var a; // Declared but not initialized
3  console.log(a); // Output: undefined
4
5  //! 2. Initialization ⇒ var: You can initialize a variable declared using var
6  var a;
7  a = 20;
8  console.log(a); // Output: 20
9
10 //! 3. Declaration and Initialization ⇒ var: You can declare and initialize at the same time
11 var x = 50;
12 console.log(x); // Output: 50
13
14 //! 4. Re-declaration (Same Variable Name) ⇒ var: You can re-declare a variable using var
15 var a = 10;
16 var a = 20; // No error
17 console.log(a); // Output: 20
18
19 //! 5. Re-initialization ⇒ var: You can re-initialize a var variable.
20 var a = 10;
21 a = 30; // Allowed
22 console.log(a); // Output: 30
23
24 //! 6. Scope ⇒ var: The scope is global or function-wide (not block-scoped)
25 if (true) {
26   var a = 10;
27 }
28 console.log(a); // Output: 10
29
30 //! 7. Hoisting ⇒ var: Variables declared with var are hoisted and initialized with undefined
31 console.log(a); // Output: undefined
32 var a = 10;
```

2. **let** ⇒

- a. The let variable is used to declare block-scoped variables, meaning the variable is accessible only within the block in which it is defined.
- b. A let variable is introduced in **ES6 (2015)** version.

- c. Variables declared with `let` are hoisted, but remain in the *Temporal Dead Zone* until initialization.
- d. For Example →

```
1  1. Declaration ⇒ let: You can declare a variable using let  
2  let b; // Declared but not initialized  
3  console.log(b); // Output: undefined  
4  
5  2. Initialization ⇒ let: You can initialize a variable declared using let  
6  let b;  
7  b = 10;  
8  console.log(b); // Output: 10  
9  
10 3. Declaration and Initialization ⇒ let: You can declare and initialize at the same time  
11 let y = 40;  
12 console.log(y); // Output: 40  
13  
14 4. Re-declaration (Same Variable Name) ⇒ let: Re-declaration is not allowed with let  
15 let b = 10;  
16 let b = 20; // Error: Identifier 'b' has already been declared  
17  
18 5. Re-initialization ⇒ let: You can re-initialize a let variable..  
19 let b = 10;  
20 b = 40; // Allowed  
21 console.log(b); // Output: 40  
22  
23 6. Scope ⇒ let: The scope is block-scoped  
24 if (true) {  
25   let b = 20;  
26 }  
27 console.log(b); // Error: b is not defined  
28  
29 7. Hoisting ⇒ let: Variables declared with let are hoisted but not initialized  
30 console.log(b); // Error: Cannot access 'b' before initialization  
31 let b = 20;
```

3. **const** ⇒

- a. The `const` variable declares block-scoped variables that cannot be updated or re-declared.
- b. A `const` variable is introduced in *ES6 (2015)* version.
- c. Variables declared with `const` are hoisted, but remain in the *Temporal Dead Zone until* initialization.
- d. For Example →

```

1  #!/ 1. Declaration ⇒ const: A const variable must be initialized at the time of declaration.
2  // Declaration without initialization throws an error
3  const c; // Error: Missing initializer in const declaration
4
5  #!/ 2. Initialization ⇒ const: You cannot reassign or initialize after declaration
6  const c = 30;
7  c = 40; // Error: Assignment to constant variable
8
9  #!/ 3. Declaration and Initialization ⇒ const: You must declare and initialize at the same time
10 const z = 60;
11 console.log(z); // Output: 60
12
13 #!/ 4. Re-declaration (Same Variable Name) ⇒ const: Re-declaration is not allowed with const
14 const c = 10;
15 const c = 20; // Error: Identifier 'c' has already been declared
16
17 #!/ 5. Re-initialization ⇒ const: Re-initialization is not allowed with const.
18 const c = 10;
19 c = 20; // Error: Assignment to constant variable
20
21 #!/ 6. Scope ⇒ const: The scope is also block-scoped
22 if (true) {
23   const c = 30;
24 }
25 console.log(c); // Error: c is not defined
26
27 #!/ 7. Hoisting ⇒ const: Variables declared with const are hoisted but not initialized
28 console.log(c); // Error: Cannot access 'c' before initialization
29 const c = 30;
30

```

4. JavaScript Scope:

- a. JavaScript Scope is the area where a variable exists and is accessible.
- b. OR
- c. In JavaScript scope refers to the accessibility and visibility of variables, functions, and objects within different parts of your code.
- d. **Types of Scopes:**
 - i. **Global Scope** → Variables declared outside of functions, blocks, or curly braces (have a global scope. -Global variables can be accessed from anywhere in the program.

- ii. **Local Scope or Function Scope** → Variables declared inside a function have local (or function) scope.
 - These variables cannot be accessed outside the function where they are declared.
 - Note:** *Variables declared within the function have a function scope.*

- iii. **Block Scope** → Introduced in **ES6** with the **let** and **const** keywords.
 - Variables declared inside curly braces (**{ }**) can only be accessed within that block.
 - Note:** *Variables declared with JS Variables var do not have block scope, meaning they are still accessible outside the block.*

- iv. **Module Scope (ES6)** → Variables declared in ES6 modules (using **import** and **export**) are scoped to the module.
 - They are not accessible outside the module unless explicitly exported.
 - Each module has its own scope, and variables and functions declared within a module are private to that module unless exported.

- v. **Script Scope** → The variables which are directly present inside the JS files.

5. Variable Hoisting:

- a. Hoisting means we can initialize the variable before its declaration.
- b. The JS engine allows programmer to access the member before its declaration is known as *Hoisting*.
- c. OR
- d. Hoisting means JavaScript moves declarations to the top of the scope before code execution.
- e. Only declarations are hoisted, not initializations.
- f. **NOTE** → In JS variable hoisting is possible in *var* variable only.
 - Variables declared with *let* and *const* are also hoisted but remain in the *Temporal Dead Zone* (TDZ) until initialization.

6. Temporal Dead Zone (TDZ):

- a. The *Temporal Dead Zone* TDZ is the period of time in JavaScript where a variable is declared using *let* or *const* but cannot be accessed because it hasn't been initialized yet.
- b. If you try to access the variable before it is initialized, you get a **ReferenceError**.

7. Comparison Between var vs let vs const

- a. Create the comparison table whatever we have discussed in the class or create the table that you have taken the picture in the class.